UCS503 – Software Engineering

Thapar Exchange

Campus Marketplace and Resource Sharing Platform

Submitted by:

1024031005 Kanishk Khinchi
1024030290 Mahesh
1024030729 Amrita

Group: 3C53, BE Third Year, C0E

Submitted to:

Dr. Paramveer Sidhu

Project Evaluation Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 4
- 1.3 Scope and Limitations . . . 4

2 Methodology and Design **5**
- 2.1 System Architecture . . . 5
  - 2.1.1 High-Level Workflow . . . 6
  - 2.1.2 Technical Stack . . . 6
- 2.2 Implementation Details . . . 7
  - 2.2.1 Authentication and Listing Management . . . 7
  - 2.2.2 Rental Booking Workflow . . . 7
  - 2.2.3 Database Design . . . 7

3 Results and Evaluation **9**
- 3.1 Environment and Database Setup . . . 9
- 3.2 Functional Test Plan . . . 9
- 3.3 Performance Evaluation . . . 10
- 3.4 Analysis . . . 11
- 3.5 Limitations . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations . . . 12
- 4.4 Final Remarks . . . 13

Chapter 1

Introduction

1.1 Background and Context

Students often need resources for a limited period, including textbooks, calculators, cycles, sports equipment, electronics, hostel essentials, and other useful items. Buying every item new can be expensive and unnecessary when a resource is needed temporarily. At the same time, students may own items that are no longer in regular use but could be useful to others.

Such exchanges are often arranged through messaging groups, personal contacts, or informal social media posts. These approaches can make it difficult to discover relevant items, compare details, confirm availability, and maintain a structured record of rental requests. Thapar Exchange, also referred to as ThaparRent in the repository README, is a campus-focused web application intended to make student-to-student resource sharing more organized.

The repository contains a React-based frontend and an Express backend connected to a PostgreSQL database through Prisma ORM. The current backend includes account registration and login, listing endpoints, and rental-booking workflows. The project is under active development; therefore, this report distinguishes implemented functionality from broader features proposed for future versions.

1.1.1 Problem Statement

Students at Thapar Institute of Engineering and Technology may need to rent or share items, but informal channels do not provide a single structured place to publish listings, browse resources, authenticate users, and manage rental requests. Information can be scattered across different groups, while requests and their statuses may be difficult to track consistently.

The project addresses this problem through a web-based campus marketplace with user accounts, item listings, and a database-backed booking process. The current backend supports registration and login, public listing retrieval, authenticated listing creation, and booking requests with owner-controlled acceptance or rejection. Institutional email verification, advanced search filters, condition documentation, reporting, and administrative moderation should not be considered complete unless verified in the current code.

1.2 Project Objectives

1. Develop a centralized web application for students to discover and offer campus resources.
2. Implement registration and login, storing password hashes rather than plaintext passwords.
3. Provide listing endpoints for item details, category, daily rental price, optional image URL, availability, and owner.
4. Implement rental requests and owner-controlled acceptance or rejection.
5. Store users, listings, and bookings in PostgreSQL using Prisma ORM.
6. Validate core workflows and document limitations before making performance, usability, or security claims.

1.3 Scope and Limitations

This report focuses on the repository's architecture, backend modules, data model, and local database setup. The observed setup results are that Prisma schema validation succeeded, Prisma Client was generated, and the existing initialization and booking migrations were applied successfully to the local PostgreSQL database. This does not independently establish that every end-to-end user workflow works.

No verified performance benchmarks, formal usability-study results, or evidence that all planned trust and moderation features are complete were supplied. These are treated as evaluation tasks or future work rather than implemented outcomes.

Chapter 2

Methodology and Design

2.1 System Architecture

The application follows a client–server architecture. The frontend uses React and Vite [**reactdocs**; **vitedocs**]. It communicates with a backend API implemented using Node.js and Express [**expressdocs**]. The backend uses Prisma ORM to access a PostgreSQL database [**prismadocs**; **postgresdocs**]. During local development, the Vite configuration proxies API requests to the backend.

Figure 2.1 presents the high-level component arrangement. Requests originate in the browser-based frontend, are handled by Express routes and middleware, and access persistent data through Prisma.

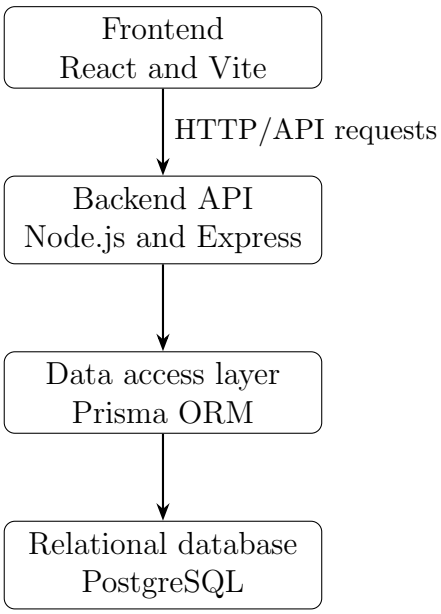


Figure 2.1: High-level architecture of Thapar Exchange

The main entities are users, listings, and bookings. A user can own multiple listings, and a listing can have multiple booking records. Each booking identifies a listing and a renter, and stores a status and timestamps. Listing states are `AVAILABLE` and `UNAVAILABLE`; booking states are `PENDING`, `ACCEPTED`, `REJECTED`, and `CANCELLED`.

2.1.1 High-Level Workflow

1. A student registers using a name, email address, and password, or logs in with existing credentials.

2. The backend validates the request, hashes new passwords with bcrypt [**bcrypt**], and issues a JSON Web Token (JWT) after successful authentication [**jwt**].

3. Public listing endpoints allow clients to retrieve listings and listing details.

4. An authenticated student creates a listing. The backend derives the owner identity from the authentication token rather than trusting an owner ID supplied in the request body.

5. A renter submits a booking request for an available listing. The backend prevents a user from requesting their own listing and rejects duplicate pending requests for the same listing by the same renter.

6. The listing owner can accept or reject a pending booking. The booking status is persisted in the database.

2.1.2 Technical Stack

Table 2.1: Technologies used in the current repository

Technology	**Role in the project**
JavaScript	Main language used by the frontend and backend.
React	Browser-based user interface.
Vite	Development server and frontend build tooling.
Node.js	Runtime for the backend API.
Express 5	HTTP routes and middleware.
PostgreSQL	Relational database for users, listings, and bookings.
Prisma ORM	Database schema, client access, and migrations.
bcrypt	Password hashing.
JSON Web Token (JWT)	Authentication tokens for protected API requests.
Git	Version control for source files and project history.